
GORGO: Online Tuning for Cross-Region Network-Aware LLM Serving

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 In cross-region LLM serving, time-to-first-token (TTFT) is highly sensitive to
2 per-request routing decisions. KV-cache reuse across replicas can eliminate much
3 of the prefill computation on multi-turn workloads, but realizing this potential
4 requires jointly balancing cache locality against replica load and network cost.
5 Additionally, public datasets to evaluate these policies do not represent long-
6 context, high prefix-reuse workloads: LMSYS and WildChat average 2 and 2.5
7 turns per conversation, 0.5 and 2.9k tokens/request, and 5 and 9% intra-user
8 reuse, respectively. To capture the setting where a majority of requests contain
9 reusable prefixes, we evaluate common routing policies such as least-load, prefix-
10 aware, session-affinity, and others on traces from a production inference endpoint
11 featuring multi-turn conversations (~ 82 requests/user), $7\text{--}45\times$ longer prompts (21k
12 avg tokens), and $2\text{--}6\times$ higher KV-cache reuse (55%). We evaluate policies on a
13 multi-region GPU cluster where cross-region network latency ranges from 26ms
14 to 466ms across replicas to test GORGO, an online policy that jointly optimizes
15 cache reuse, network latency, and queue depth to minimize p95 TTFT with no
16 offline profiling. GORGO achieves the lowest TTFT at every percentile (p50, p95,
17 p99) across both a tuning window and a held-out evaluation window against all
18 other load balancing policies, highlighting the impact of online joint optimization.

19 1 Introduction

20 Modern LLM serving is increasingly cross-region. Operators distribute model replicas across multiple
21 geographies for three reasons: to follow diurnally shifting demand from a globally distributed user
22 base, to provide redundancy against regional capacity loss or partition, and to aggregate accelerator
23 capacity that no single region currently supplies [Xia et al., 2025, Mao et al., 2025]. Once replicas
24 span regions, the gateway that routes incoming requests becomes a first-class component of time-to-
25 first-token (TTFT), sitting between the client and the inference engines and choosing one replica per
26 request based on observable state.

27 TTFT on a single replica decomposes into three terms: the network round-trip from the gateway to
28 that replica, the queueing delay behind in-flight requests admitted via continuous batching [Yu et al.,
29 2022], and the prefill computation for whatever portion of the input prompt is not already cached.
30 Modern inference engines expose a per-replica prefix-keyed KV-cache (radix-tree-indexed in SGLang,
31 block-hash-keyed in vLLM) that lets incoming requests skip prefill for any prefix already present
32 on the handling replica [Zheng et al., 2024b, Kwon et al., 2023]. On long-context multi-turn traffic
33 that saving can be the difference between sub-second TTFT and multi-second TTFT, so KV-cache
34 locality at the routing layer is a first-order determinant of latency [Xia et al., 2025, Qin et al., 2024].

35 These three TTFT terms pull in different directions. Routing to the geographically nearest replica
36 minimizes network RTT but ignores cache state; routing to the most cache-warm replica maximizes

prefix reuse but can pile onto a saturated batch; routing to the least-loaded replica balances queue depth but is cache- and locality-blind. Standard production heuristics implement one of these signals at a time: least-connections-style load balancers (NGINX) [F5 NGINX, 2024], AIBrix-style longest-prefix-match [AIBrix Team, 2024], consistent-hash session affinity [Karger et al., 1997], and so on. They suffice for workloads with short prompts, weak prefix reuse, and uniform replica latencies, but interactive long-context traffic breaks all three assumptions: production chat traces concentrate most tokens in a small set of returning users with substantial prefix overlap, prompts run to tens of thousands of tokens, and cross-region RTTs differ by an order of magnitude. The right routing target for a given request can change minute-to-minute as load shifts, and weighting one signal in isolation leaves performance on the table at the tail.

A second obstacle is empirical: the public chat datasets used in prior LLM-serving evaluations underrepresent the regime where these trade-offs matter. LMSYS-Chat-1M [Zheng et al., 2024a] and WildChat-4.8M [Zhao et al., 2024] average 470 and 2,900 tokens per request with intra-user prefix reuse below 10%; the production endpoint we evaluate carries 21,000 tokens per request on average and 53.7% intra-user prefix reuse. Public datasets thus produce TTFT distributions that are nearly indistinguishable across cache-aware and cache-blind policies in our measurements (§7), masking routing differences that production traffic exposes.

Contributions. (i) A workload characterization (§3) placing three datasets on the same prefix-reuse and request-length axes; the GLM-5.1 production trace has 7–45× longer prompts and 2–6× higher token-weighted reuse than LMSYS-Chat-1M and WildChat-4.8M. (ii) A routing cost model and three variants collectively called GORGO (§4) that score each replica by an additive combination of EWMA network RTT, estimated prefill cost over uncached tokens, and projected decode-rate cost over running and queued tokens. Variants differ only in how the two per-token rate parameters are set: a static configuration, a per-replica median-of-rates fit, or a Gaussian (1+1)-ES hill climb that directly minimizes p95 TTFT. (iii) An experimental setup (§6) running each of eight policies on its own dedicated 2×L40S three-replica fleet in Seoul, Frankfurt, and Ashburn, with cross-region RTTs spanning 26–466 ms. (iv) Empirical results (§7) across multiple 30-minute slices of GLM-5.1 traffic on two consecutive days plus a higher-concurrency stress window: in every window, two GORGO variants together cover the lowest TTFT at p50, p95, and p99 against seven baseline policies; on a WildChat replay the GORGO advantage collapses, confirming that the result is regime-dependent on long-context, high-reuse traffic.

2 Background

KV caching and prefix reuse. Inference engines such as SGLang [Zheng et al., 2024b] and vLLM [Kwon et al., 2023] maintain a Key–Value cache of attention activations for every active sequence; new tokens are appended to this cache step-by-step and admitted into the next decoding iteration via continuous batching [Yu et al., 2022]. Beyond per-request state, the same cache enables *prefix reuse*: when an incoming request shares a token-level prefix with a sequence already cached on a replica, that prefix’s KV state can be reused and prefill is skipped for those tokens. SGLang’s RadixAttention [Zheng et al., 2024b] operationalizes this via a per-replica radix trie that tracks which prefixes are resident; on multi-turn workloads where the same user returns repeatedly, a replica that has seen the conversation can serve the next turn at a fraction of the prefill cost.

Request-level routing as its own layer. Inside a single replica, a per-request scheduler decides admission and order within the running batch [Zheng et al., 2024b, Yu et al., 2022]; across replicas, a *request-level router* (or gateway) decides which replica receives a request in the first place. AIBrix [AIBrix Team, 2024] is a recent example of such a routing layer designed for LLM gateways, exposing prefix-cache, least-load, and least-request policies behind a common interface; production gateways like NGINX [F5 NGINX, 2024] and AWS Bedrock cross-region inference [Amazon Web Services, 2026] occupy the same architectural slot but at the network level, with no surface for KV state. Routing-layer decisions matter disproportionately on long-context workloads because the prefill term they implicitly select for can dominate TTFT [Xia et al., 2025, Qin et al., 2024].

Routing-cost decomposition. For request r routed to replica u , TTFT depends on three signal families the gateway can observe, each of which decomposes further: (1) *Network latency*: the proxy-to-replica RTT $\text{rtt}(u)$. (2) *Replica load*, comprising (2a) decode tokens of running requests in

the current batch, (2b) prefill tokens of requests queued ahead of r , and (2c) r 's own prefill cost once admitted. (3) *Available prefix cache*, comprising prefix overlap with (3a) the running batch (some KV state may already be evicted), (3b) queued requests (their KV state will exist by the time r runs), and (3c) r 's own cached prior turns under the same conversation. A routing policy is a function over these nine sub-signals; no production policy we are aware of jointly weighs even all three top-level families.

Common policies. We consider five non-GORGO baselines representative of production systems: `random` (cache- and load-blind), `least-request` and `least-load` (load-aware, cache-blind), `prefix-cache` (longest-prefix-match with a `least-request` fallback under load imbalance, in the AIBrix style), and `simple-session-affinity` (token-prefix hash to a fixed replica). §5 states their exact form. None of the five takes more than one of the three signal families above as a primary objective.

Production cross-region gateways and prior systems. Industrial cross-region routing exists today but is opaque to KV state. AWS Bedrock cross-region inference [Amazon Web Services, 2026] and GKE multi-cluster Gateways [Google Cloud, 2026] provide global routing and failover but treat inference as an HTTP endpoint. Serverless GPU platforms such as Modal [Modal, 2026] support intra-region prefix caching but do not optimize across regions; SkyPilot [Yang et al., 2023] aggregates capacity across clouds but does not observe per-replica KV state. SkyServe [Mao et al., 2025] manages cross-region placement and SkyWalker [Xia et al., 2025] adds KV-aware cross-region load balancing with selective KV pushing. GORGO occupies the gap between opaque gateways and KV-aware single-region schedulers: a per-request, multi-region routing layer that combines all three signal families on a single scalar cost score, with the two scaling weights tuned online.

3 Dataset

Public datasets sit in the wrong regime. Routing policies that exploit cache locality or user stickiness improve TTFT by choosing a replica with high prefix-cache reuse, which we measure as the prefix-hit rate—the fraction of an incoming request's input tokens that match a previously-seen prefix on the chosen replica. On a corpus where prefix-hit rates are uniformly low across all replicas, no cache-aware policy has anywhere to win. This is the regime occupied by the public chat datasets: LMSYS-Chat-1M [Zheng et al., 2024a] averages 467 input tokens per request and 9% global token-weighted prefix reuse, and WildChat-4.8M [Zhao et al., 2024] averages 2,925 input tokens and 5.3% intra-user reuse. The combination of short prompts and weak reuse pushes prefill cost down enough that TTFT distributions are nearly indistinguishable across cache-aware and cache-blind policies (we confirm this in §7 via a WildChat replay). Public benchmarks on these corpora consequently underestimate the gain from joint cache-network-queue-aware routing.

GLM-5.1 production trace. To measure routing in the regime where the trade-offs actually arise, we compile a long-context, multi-turn chatbot dataset from production inference traces of a commercial inference provider's GLM-family endpoint (the deployed variant is GLM-5.1; see [GLM Team, 2024] for the most recently published technical report from the model family). The corpus contains 411,169 requests over 7 days from 4,984 distinct users, yielding ~82 requests per user on average, with an average input length of 21,043 tokens and 53.7% intra-user token-weighted prefix reuse. Reuse is measured with a token-weighted prefix trie that ingests each request's tokenized message sequence in arrival order, partitioned by user, and reports intra-user savings, cross-user (additional) savings, and global savings. Table 1 places the three datasets on the same axes. LMSYS-Chat-1M's release reports 210,479 unique client IPs across its 1M conversations [Zheng et al., 2024a], but the publicly distributed dataset schema does not include the per-conversation IP column, so our prefix-trie analysis can only partition LMSYS by conversation ID; we cite the HF-published user count and leave intra-user reuse unmeasured here.

Three contrasts (Table 1, Fig. 1) matter for routing. *Request length:* GLM-5.1 prompts are 45× longer than LMSYS and 7× longer than WildChat—at the prefill rates we measure, a 21,000-token uncached prompt costs seconds, not milliseconds. *User concentration:* GLM-5.1 averages 82 requests per user; WildChat 1.7; LMSYS 4.8. Both public sets are dominated by single-shot or near-single-shot users—intra-user prefix reuse depends on returning users issuing many requests, which neither public set has at the scale of a long-running production endpoint. *Reuse composition:* GLM-5.1's reuse is intra-user multi-turn dialogue (53.7% intra plus 1.6% additional cross-user), while WildChat's 34.4%

Table 1: Dataset comparison along axes that determine the routing-decision surface. Token counts use the same byte-pair tokenizer in all rows. Reuse percentages are token-weighted prefix-trie savings. *LMSYS user count is from the dataset card [Zheng et al., 2024a] (210K unique IPs across 1M conversations); the public schema does not expose the IP column, so we cannot recompute its intra-user reuse here and leave that cell as “—”.

Dataset	Total reqs	Users	Reqs/user	Avg in. tokens	Intra-user reuse	Global reuse
LMSYS-Chat-1M	1,000,000	210,479*	4.8	467	—*	9.0%
WildChat-4.8M	3,199,860	1,833,730	1.7	2,925	5.3%	34.4%
GLM-5.1 production	411,169	4,984	82.5	21,043	53.7%	55.3%

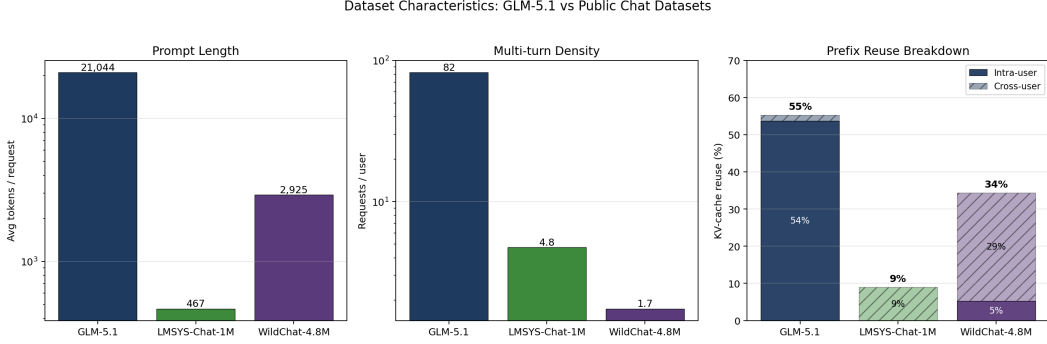


Figure 1: Three axes of dataset characterization. *Left*: average input tokens per request (log scale)—GLM-5.1 is 45× longer than LMSYS and 7× longer than WildChat. *Center*: requests per user on a log scale—82 on GLM-5.1 vs. 4.8 on LMSYS-Chat-1M vs. 1.7 on WildChat-4.8M. *Right*: prefix-reuse breakdown—GLM-5.1 reuse is dominated by intra-user repeat-conversation prefixes; WildChat reuse is dominated by cross-user structural similarity (shared system prompts, viral templates); LMSYS’s intra-user component is not measurable on the public schema (no IP column), so only its global savings is plotted.

145 global is mostly cross-user shared structure (5.3% intra). On GLM-5.1, a policy that lands a returning
 146 user on a replica that has seen their conversation can skip more than half of the prefill; on the public
 147 datasets, the same policy has nothing to skip.

148 **Trace format and filtering.** We re-encode GLM-5.1 into the Mooncake FAST’25 trace schema
 149 [Qin et al., 2024] so that benchmarking is time-sequenced and reproducible: each trace is a JSONL
 150 file in which every row carries an absolute-millisecond timestamp, request metadata (input/output
 151 token counts, target model), and the already-tokenized request body, which avoids exposing user data.
 152 Requests with >24,000 input tokens are filtered out; this is short of the model’s 32k native context
 153 window to leave headroom for our tokenizer disagreeing with the source tokenizer (so as to shrink
 154 the failed-request surface during testing). The filtered trace is then chunked by absolute timestamp
 155 into 30-minute windows; the windows we benchmark in §7 are drawn from two consecutive days
 156 of this trace at the same time-of-day band. Replay is open-loop: requests are issued at the recorded
 157 arrival timestamps regardless of in-flight load, exposing each policy to production burstiness rather
 158 than to a closed-loop generator that would mask queueing pathologies.

159 4 GORGO

160 **Cost model.** Let r denote an incoming request with input token sequence x_r , and let u denote a
 161 candidate replica with KV-cached token sequences c_u currently resident in its prefix-keyed KV cache.
 162 Following the decomposition in §2, the GORGO score for routing r to u is the sum of the three TTFT
 163 terms the gateway can observe:

$$\text{score}(u, r) = \text{rtt}_u + t_{\text{prefill}} \cdot \max(0, |x_r| - |x_r \cap c_u|) + \text{queued_tokens_weight} \cdot (\text{queued_tokens}_u + \text{used_kv_tokens}_u). \quad (1)$$

164 The proxy sends r to the replica with the minimum score. The three signals are sourced as follows.
 165 *Network latency.* rtt_u is the proxy-to-replica round-trip from a sidecar probe that pings every replica

every 30 s; latency is smoothed with an exponentially-weighted moving average and stored decoupled from the Prometheus metrics scrape so handler contention does not pollute the estimate. *Replica load*. queued_tokens_u comes from a proxy-side counter the proxy maintains per replica; used_kv_tokens_u comes from SGLang’s `num_used_tokens` Prometheus metric. Their sum is the token-weighted backlog on u (queued tokens plus the live KV-cache footprint of running requests). *Available prefix cache*. $|x_r \cap c_u|$ is the count of input tokens of r already cached on u , computed by a proxy-side global radix trie over per-replica cached prefixes; the trie is updated once per request completion and queried once per routing decision.

The two cost-function parameters t_{prefill} and $\text{queued_tokens_weight}$ have units of seconds per token. Their *scope* differs by variant: `gorgo-static` and `gorgo-autotune` keep separate values per replica, so heterogeneous fleets can carry per-replica calibration; `gorgo-hillclimb` keeps a single global pair shared across all replicas, holding the search space two-dimensional and accelerating online convergence.

Three weight-selection variants. The variants share Equation 1 and differ only in how the two parameters are chosen.

`gorgo-static` fixes both parameters at run start and never adjusts them. Starter values can be hand-set or carried over from a prior tuning window; the variant isolates the contribution of the cost model independent of online adaptation.

`gorgo-autotune` re-fits both rates online by linear regression through the origin on a trailing 64-sample window, partitioned by destination replica, every 16 new samples. For $t_{\text{prefill}}(u)$ it fits network-latency-normalized TTFT ($\text{TTFT} - \text{rtt}_u$) as a linear function of uncached input tokens; for $\text{queued_tokens_weight}(u)$ it fits network-latency-normalized total completion time as a linear function of completion tokens. The variant treats the cost-model weights as identifiable physical rates, which is also its failure mode—the per-sample prefill rate inflates when the uncached count is small (high cache hit), so the through-origin slope drifts upward.

`gorgo-hillclimb` runs as a sidecar tuner that treats the two global parameters as black-box knobs and minimizes negative p95 TTFT over a rolling 64-request window with a Gaussian (1+1)-evolution strategy [Rechenberg, 1973, Beyer and Schwefel, 2002]: one parent and one offspring per iteration, with selection only over offspring. Initial values are sampled from $[10^{-3}, 10^{-1}]$. Every 16 new samples the tuner scores the current incumbent, accepts or rejects the most recent candidate, and proposes a new candidate $\theta_{\text{new}} = \exp(\log \theta_{\text{best}} + \sigma \mathcal{N}(0, I))$, merging accepted parameters into the live GORGO policy. Step size σ adapts via Rechenberg’s 1/5-success rule [Rechenberg, 1973, Beyer and Schwefel, 2002]: of the last 5 candidates evaluated, roughly one should improve the incumbent. We track the acceptance rate m over a trailing window and adjust σ multiplicatively—decreasing it (factor ~ 0.82) when $m > 1/5$ (search too tight) and increasing it (factor $\sim 1/0.82$) when $m < 1/5$ (search too loose). On a 30-minute window with a 64-sample cohort and 16-sample hop, this drives the search to within a few percent of the best-found incumbent in roughly a hundred candidate evaluations (Fig. 2).

5 Baselines

We compare GORGO against five baseline policies, each chosen because it isolates one signal that production gateways already deploy: `random` as the lower bound, `least-request` for request-count load (the LLM-gateway analogue of NGINX `least_conn`), `least-load` for token-weighted load, `prefix-cache` for KV locality (the AIBrix configuration), and `simple-session-affinity` for static hash-based locality. GORGO attempts to subsume all five via Equation 1; the per-percentile patterns in §7 show which signal each baseline implicitly weights and where ignoring the others costs latency.

`random` samples a replica uniformly per request; it is the lower bound.

`least-request` routes to the replica with the fewest in-flight requests, scoring by $\max(\text{sgLang_running_reqs}_u, \text{proxy_inflight}_u)$ to bridge the ~ 10 s staleness between metrics scrapes; this is the LLM-gateway analogue of NGINX’s `least_conn` [Mitzenmacher, 2001].

`least-load` routes to the replica with the lowest aggregate token-weighted load: $\text{running} + \text{queued} + \text{used_kv_tokens} + \text{queued_tokens}_u$. It is heavier-signal than `least-request` but cache-blind.

Table 2: GLM-5.1 W1 (tuning window, $c=32$). TTFT in seconds. Bold is best in column. Completed counts from proxy logs; success rate $\approx 84.2\text{--}84.6\%$.

Policy	Completed	p50	p95	p99	max	Succ. %
gorgo-hillclimb	4,065	0.288	1.466	2.307	4.39	84.4
simple-session-affinity	4,054	0.329	1.482	2.804	14.96	84.2
gorgo-static	4,076	0.299	1.605	2.357	4.16	84.6
least-request	4,074	0.293	1.632	2.355	13.71	84.6
prefix-cache	4,075	0.297	1.654	3.017	5.96	84.6
gorgo-autotune	4,076	0.294	1.658	2.428	4.18	84.6
least-load	4,066	0.297	1.728	2.629	4.64	84.4
random	4,076	0.296	1.828	3.171	28.99	84.6

prefix-cache is the AIBrix-style longest-prefix-match policy [AIBrix Team, 2024]. It picks the replica with the longest cached prefix match and falls back to **least-request** when the running-request imbalance exceeds a soft threshold.

simple-session-affinity hashes the first 256 token IDs of the prompt modulo the number of replicas [Karger et al., 1997]. The same prefix always maps to the same replica, maximizing intra-user reuse but providing no load-escape mechanism.

6 Experimental Setup

Hardware and software. Each policy runs on its own dedicated three-replica fleet so routing decisions for one policy cannot warm or cool another policy’s caches. Each fleet consists of one 2×L40S SGLang [Zheng et al., 2024b] server (tensor-parallel 2, 96 GB VRAM per replica) in each of Asia (Seoul, `ap-seoul`), Europe (Frankfurt, `eu-frankfurt`), and US East (Ashburn, `us-ashburn`), serving Qwen3.5-35B-A3B-FP8 [Qwen Team, 2025]. A CPU proxy in US East hosts the routing policy and replays the trace. The eight policies (**random**, **least-request**, **least-load**, **prefix-cache**, **simple-session-affinity**, and the three GORGO variants) are benchmarked simultaneously on disjoint fleets so that any cross-region network variance during a window applies equally to all of them; GPUs are not shared across policies.

The proxy scrapes Prometheus metrics every ~ 10 s and runs a separate EWMA-smoothed HTTP probe for per-replica RTT, decoupled from the metrics scrape to avoid handler contention. Before each run a homogeneity check issues 16 streaming requests directly to each replica; runs whose worst-to-median TTFT ratio exceeds $3\times$ are flagged (none occurred).

Windows. We benchmark across three disjoint 30-minute slices of the GLM-5.1 trace: **W1** (tuning, $c=32$): GORGO variants seed from manually chosen starter weights and the online tuners adapt during the window. **W2** (cross-day evaluation, $c=32$): same time-of-day band as W1 on a different day; **gorgo-static** deploys the per-replica weights that **gorgo-hillclimb** learned in W1 and never adjusts them, while the other GORGO variants continue tuning. **Stress** ($c=64$): the W2 trace replayed at $2\times$ the in-flight-request cap to test behavior under elevated load. Concurrency c is the in-flight-request cap at the proxy. Output is capped at 128 tokens per request so TTFT, not decode, dominates the latency budget.

Cross-region RTT. Probed from the US East proxy with a dedicated lightweight HTTP probe over the duration of W1, mean EWMA-smoothed RTT is 25.6 ± 3.8 ms to the local US East replica, 232 ± 72 ms to Frankfurt, and 466 ± 138 ms to Seoul; coefficient of variation on the cross-region legs is 29–31%, reflecting routing-table churn within the window.

Metrics. For each policy and window we report TTFT p50, p95, and p99 over all successful streaming responses. Failed requests (HTTP 4xx/5xx, dominated by prompt-too-long at the effective context boundary) are excluded from percentiles but counted in success rate.

7 Results

In W1 (Table 2), **gorgo-hillclimb** delivers the lowest p50 (0.288 s), p95 (1.466 s), and p99 (2.307 s)—a single GORGO variant sweeps every percentile during tuning, with a 19.8% reduction

Table 3: GLM-5.1 W2 ($c=32$, held-out evaluation: 30-minute window immediately following W1). All GORGO variants are initialized with the hyperparameters `gorgo-hillclimb` ended W1 at ($t_{\text{prefill}}=0.983$, $\text{queued_tokens_weight}=4.4 \times 10^{-4}$); `gorgo-static` freezes them, the others continue tuning. Bold is best in column.

Policy	Completed	p50	p95	p99	max	Succ.%
gorgo-static	3,573	0.294	1.510	2.791	15.26	84.9
gorgo-hillclimb	3,583	0.334	1.535	2.595	12.68	85.1
prefix-cache	3,578	0.316	1.632	2.860	34.34	85.0
least-request	3,579	0.303	1.757	2.663	5.22	85.1
random	3,575	0.336	1.763	2.612	30.93	85.0
least-load	3,580	0.363	1.813	2.659	3.84	85.1
gorgo-autotune	3,578	0.368	1.888	3.001	5.70	85.0
simple-session-affinity	3,570	0.396	2.080	4.917	36.60	84.8

Table 4: GLM-5.1 Stress, nighttime window ($c=64$). Same time-of-day band as W1, day after W1. `simple-session-affinity` did not complete this sweep and is omitted. Bold is best in column.

Policy	Completed	p50	p95	p99	max	Succ.%
gorgo-hillclimb	2,466	0.281	1.331	1.832	3.05	89.5
prefix-cache	2,468	0.288	1.394	1.992	2.95	89.6
gorgo-static	2,472	0.179	1.427	2.302	4.06	89.8
least-request	2,465	0.281	1.605	2.268	4.16	89.5
gorgo-autotune	2,469	0.280	1.690	2.298	18.84	89.7
least-load	2,465	0.281	1.700	2.290	2.98	89.5
random	2,460	0.287	1.724	2.357	4.00	89.3

in p95 TTFT versus the `random` baseline (1.466 s vs. 1.828 s). `gorgo-static` (starter weights, never adjusted) already beats every non-GORGO policy on p99 and ranks third on p95, showing the cost model alone improves on single-signal heuristics; `gorgo-hillclimb` extends the lead by adapting global rates online. `prefix-cache` is competitive at p95 but tail-degraded at p99 (3.017 s) by concentrating traffic onto warm-cache replicas; `simple-session-affinity`’s p99 (2.804 s) and max (14.96 s) reflect transient overload of the hashed replica. Starting from $t_{\text{prefill}} = 0.07$ and $\text{queued_tokens_weight} = 0.06$ (autotuned over the preceding 30-minute window), the ES converged to $t_{\text{prefill}} \approx 0.983$ and $\text{queued_tokens_weight} \approx 4.4 \times 10^{-4}$: the high t_{prefill} reflects $\sim 9.5\text{k}$ uncached tokens per request after 55% reuse, and the low $\text{queued_tokens_weight}$ reflects that deep queues are rare at $c=32$.

In W2 (Table 3), `gorgo-static`—deploying the W1-learned hyperparameters without adjustment—wins p50 (0.294 s) and p95 (1.510 s); `gorgo-hillclimb` wins p99 (2.595 s). Two GORGO variants together sweep all three percentiles, demonstrating that hillclimb-tuned values transfer cleanly across consecutive 30-minute windows. `gorgo-static` beats `gorgo-hillclimb` on p50/p95 because the W1-learned weights are near-optimal and continuing to tune under a stationary objective adds proposal variance—an operator can run `gorgo-hillclimb` during a tuning window and freeze the result into `gorgo-static` for production.

Stress at $c=64$. To stress-test the W2 result we increase the in-flight-request cap from 32 to 64 and benchmark two further 30-minute windows: a nighttime window on the day after W1 (Table 4) and a daytime window on W1 itself (Table 5). Both seed all GORGO variants with the W1-learned hyperparameters.

In the nighttime stress window (Table 4), `gorgo-static` wins p50 and `gorgo-hillclimb` wins p95 and p99 (1.331 s, 1.832 s; respectively 6.0% and 13.5% below the hillclimb’s $c=32$ W2 result, as the ES adapts to deeper queues). The two together sweep all three percentiles.

In the daytime stress window (Table 5) the picture shifts. `gorgo-static` now wins both p50 (0.193 s) and p95 (1.400 s); `prefix-cache` wins p99 (1.973 s). `gorgo-hillclimb`’s p99 inflates to 7.186 s—under midday traffic the live (1+1)-ES occasionally proposes weights that produce a heavy-tailed minute, and the negative-p95 objective does not penalize that excursion fast enough. The practical implication is the same one W2 already suggested: freeze hillclimb-tuned weights for production

Table 5: GLM-5.1 Stress, daytime window ($c=64$, same day as W1, midday traffic regime). Bold is best in column.

Policy	Completed	p50	p95	p99	max	Succ. %
gorgo-static	3,372	0.193	1.400	2.140	3.37	89.6
prefix-cache	3,376	0.295	1.416	1.973	3.75	89.7
simple-session-affinity	3,375	0.324	1.531	2.487	28.07	89.7
gorgo-hillclimb	3,376	0.226	1.581	7.186	38.29	89.7
least-request	3,380	0.275	1.694	2.440	20.07	89.8
random	3,371	0.309	1.700	2.421	3.99	89.6
least-load	3,374	0.309	1.735	2.419	16.91	89.7
gorgo-autotune	3,377	0.299	1.748	2.519	23.63	89.7

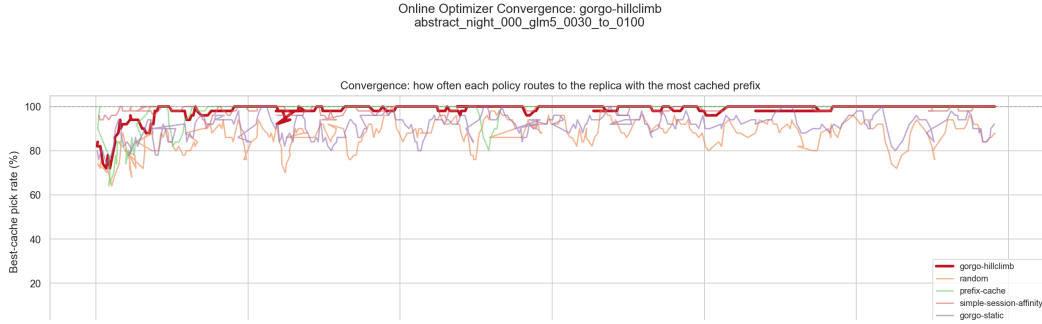


Figure 2: Best-cache-pick rate over the W1 tuning window for five policies. **gorgo-hillclimb** (red) converges to ~ 95 – 100% within the first few minutes; baselines either oscillate or stay flat at lower rates.

284 (gorgo-static), and reserve gorgo-hillclimb for tuning windows where its exploration can be
285 tolerated.

286 **Online-tuning convergence.** Figure 2 plots the best-cache-pick rate—the fraction of requests
287 routed to the replica that already holds the most of the request’s input prefix—over the W1 tun-
288 ing window for five policies. **gorgo-hillclimb** converges to picking the best-cached replica on
289 ~ 95 – 100% of requests within the first ~ 2 minutes of online tuning and stays there for the remain-
290 der of the window; the other policies either oscillate (**prefix-cache**) or remain at lower rates
291 (**simple-session-affinity**, **random**, **gorgo-static**). This is the empirical signature of the
292 $(1+1)$ -ES with the $1/5$ rule: fast initial descent during the first few hop cycles, then stable refinement.

293 **Regime sensitivity.** A control replay on WildChat-4.8M with the same fleet and proxy re-
294 verses the policy ranking: at average prompt length 2,925 tokens and 5.3% intra-user reuse,
295 **simple-session-affinity** wins p95 (1.025 s) and p99 (1.712 s), and the GORGO variants are
296 not competitive (**gorgo-hillclimb** 1.325 s p95; **gorgo-static**’s GLM-5.1-trained weights are
297 particularly mis-calibrated). The advantage GORGO shows on GLM-5.1 is regime-dependent on
298 long-context, high-reuse traffic; the full WildChat table and analysis are in Appendix A.

299 8 Limitations

300 **Statistical scope.** Four 30-minute windows from one production trace (W1 tuning, W2 held-out,
301 two stress windows at $c=64$); each policy run once per window. With $\sim 2,500$ – $4,800$ successful
302 responses, p99 standard error is $\sigma_{\text{TFT}} / \sqrt{n} \cdot 0.01 \cdot 0.99 \approx 0.05$ – 0.07 s. GORGO p95/p99 wins are
303 several SE clear of the next baseline in W2 and the nighttime stress; in W1 the p99 lead is ~ 1 SE;
304 on the daytime stress **gorgo-hillclimb**’s p99 inflates and the win goes to **prefix-cache** instead.
305 Cross-window replication for confidence intervals on every percentile remains future work. **ES tail**
306 **under non-stationary load.** **gorgo-hillclimb**’s p99 spikes on the daytime stress window (7.186 s
307 vs. **gorgo-static**’s 2.140 s on the same window; Table 5). The negative-p95-TTFT objective does
308 not penalize a heavy-tailed minute fast enough for the $(1+1)$ -ES to back out of a bad candidate.
309 Frozen weights from a tuning window (**gorgo-static**) are the safer production setting; live tuning is

appropriate for calibration runs only. **Regime sensitivity.** The WildChat replay (§7) shows GORGO’s advantage is specific to long-context, high-reuse traffic: on prompts averaging 2,900 tokens with 5.3% intra-user reuse, the prefill term is small, the per-token rates do not converge in 30 minutes, and **simple-session-affinity** wins. We claim improvement in the regime characterized in §3, not universally. **Methodological and operational caveats.** `gorgo-autotune` is the weakest GORGO variant on GLM-5.1 because the per-sample rate (TTFT – rtt)/uncached inflates at high cache hit, biasing the fit. `gorgo-hillclimb` requires several hop cycles to converge; this fits comfortably in 30-minute windows but not in arbitrarily short ones. Each policy ran on a single-tenant three-replica fleet; the centralized proxy is a single point of failure, and a distributed peer-summary GORGO variant is sketched but not benchmarked. The fleet uses homogeneous GPUs and does not split prefill/decode [Zhong et al., 2024, Patel et al., 2024], which would require an additional routing term; GORGO uses only HTTP-exposed metrics, not scheduler-internal state [Pan et al., 2025, Yang et al., 2025a] that could tighten the prefill estimate.

9 Related Work

Prefix caches and reuse. RadixAttention in SGLang [Zheng et al., 2024b] stores per-request KV state in a radix tree; PagedAttention in vLLM [Kwon et al., 2023] enables prefix sharing through paged memory. KVLink [Yang et al., 2025b], ChunkKV [Liu et al., 2025], KVFlow [Pan et al., 2025], and Learned Prefix Caching [Yang et al., 2025a] improve the single-replica cache but do not decide which replica serves a request. **Routing and scheduling.** Mooncake [Qin et al., 2024] is a KV-cache-centric architecture and the source of our trace format. AIBrix [AIBrix Team, 2024] provides prefix-aware load balancing as a baseline; Preble [Srivatsa et al., 2025] is an earlier distributed prefix-aware scheduler combining locality with load-aware reassignment; k-LPM [Dexter et al., 2025] formulates LLM scheduling under TTFT constraints as NP-hard; DLPM [Cao et al., 2025] targets fairness with locality; Llumnix [Sun et al., 2024] migrates requests across replicas; Multi-LLM routers [Wu and Silwal, 2025] address model selection; CacheBlend [Yao et al., 2024] fuses cached chunks for RAG by overlap but does not consider network RTT. GORGO commits to a single scalar score per replica with online weight tuning and requires no engine modifications. **Online and learned routing.** Performance-aware LLM load balancing [Jain et al., 2025] learns a routing policy over mixed prefill/decode workloads; bandit [Auer et al., 2002] and RL schedulers [Mao et al., 2019] adapt placement from observed outcomes; Bayesian-optimization configuration tuners [Alipourfard et al., 2017] explore the broader space. GORGO keeps the policy interpretable (an additive cost) and applies (1+1)-ES only to two scaling weights, with TTFT as direct reward. **Prefill/decode batching and disaggregation.** Sarathi-Serve [Agrawal et al., 2024] uses chunked prefill to share a single batch between phases; DistServe [Zhong et al., 2024] and Splitwise [Patel et al., 2024] disaggregate prefill and decode onto separate batch queues or GPU pools. Both lines reduce the compute component of TTFT but do not resolve geo-routing trade-offs when locality and network latency conflict; GORGO is complementary and assumes an underlying stack that may already use either.

10 Conclusion

We characterize a workload regime—long-context, multi-turn, high prefix-reuse production chat traffic—where standard LLM-serving routing heuristics leave performance on the table because they optimize one of network latency, replica load, or cache locality and ignore the other two. A workload of this shape is what production cross-region inference gateways increasingly serve, and what public chat datasets do not capture (§3); evaluating routing on those datasets understates what is at stake. On a 411,000-request production trace where 53.7% of input tokens are part of a previously-seen prefix from the same user, GORGO—a single-score additive cost model with two per-token rate parameters tuned online—collectively achieves the lowest TTFT at every percentile (p50, p95, p99) on a tuning window and the immediately following held-out window, with a 19.8% reduction in p95 TTFT versus the random baseline during tuning, on a 2×L40S three-region fleet with cross-region RTTs spanning an order of magnitude. Across two higher-concurrency stress windows, frozen tuning-window weights (`gorgo-static`) win p50/p95 reliably and the live-tuned variant trades exploration risk on heavy-tailed minutes of daytime traffic—suggesting an operational pattern of running the online tuner during a calibration window and freezing the result for production. A control replay on a low-context corpus reverses the ranking, confirming the result is regime-dependent (Appendix A). The cost model and tuner fit on a stock HTTP gateway and require no offline profiling.

References

- A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, R. Ramjee, and A. Tumanov. Sarathi-Serve: Llm inference made efficient and fair. *arXiv preprint*, 2024.
- AIbrix Team. AIbrix: An open-source infrastructure for LLM inference at scale. <https://github.com/vllm-project/aibrix>, 2024.
- Omid Alipourfard, Hongqiang Harry Liu, Jianshu Chen, Shivaram Venkataraman, Minlan Yu, and Ming Zhang. CherryPick: Adaptively unearthing the best cloud configurations for big data analytics. In *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI '17)*, 2017.
- Amazon Web Services. Increase throughput with cross-region inference. Documentation, 2026. Accessed 2026-01-28.
- Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002. doi: 10.1023/A:1013689704352.
- Hans-Georg Beyer and Hans-Paul Schwefel. Evolution strategies—a comprehensive introduction. *Natural Computing*, 1(1):3–52, 2002. doi: 10.1023/A:1015059928466.
- Shiyi Cao, Yichuan Wang, Ziming Mao, Pin-Lun Hsu, Liangsheng Yin, Tian Xia, Dacheng Li, Shu Liu, Yineng Zhang, Yang Zhou, Ying Sheng, Joseph Gonzalez, and Ion Stoica. Locality-aware fair scheduling in LLM serving. *arXiv preprint arXiv:2501.14312*, 2025.
- Gregory Dexter, Shao Tang, Ata Fatahi, Qingquan Song, Tejas Dharamsi, and Aman Gupta. LLM query scheduling with prefix reuse and latency constraints. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- F5 NGINX. NGINX documentation: Load balancing methods. <https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/>, 2024.
- GLM Team. GLM-4: A family of open multilingual multimodal chat LLMs, 2024.
- Google Cloud. About multi-cluster gateways. Documentation, 2026. Accessed 2026-01-28.
- Kunal Jain, Anjaly Parayil, Ankur Mallick, Esha Choukse, Xiaoting Qin, Jue Zhang, Íñigo Goiri, Rujia Wang, Chetan Bansal, Victor Rühle, Anoop Kulkarni, Steve Kofsky, and Saravan Rajmohan. Performance aware LLM load balancer for mixed workloads. In *Proceedings of the 5th Workshop on Machine Learning and Systems (EuroMLSys '25)*, 2025.
- David Karger, Eric Lehman, Tom Leighton, Rina Panigrahy, Matthew Levine, and Daniel Lewin. Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the world wide web. In *Proceedings of the 29th Annual ACM Symposium on Theory of Computing (STOC '97)*, 1997.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- Xiang Liu, Zhenheng Tang, Peijie Dong, Zeyu Li, Yue Liu, Bo Li, Xuming Hu, and Xiaowen Chu. ChunkKV: Semantic-preserving KV cache compression for efficient long-context LLM inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. Learning scheduling algorithms for data processing clusters. In *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM '19)*, 2019.
- Ziming Mao, Tian Xia, Zhanghao Wu, Wei-Lin Chiang, Tyler Griggs, Romil Bhardwaj, Zongheng Yang, Scott Shenker, and Ion Stoica. SkyServe: Serving ai models across regions and clouds with spot instances. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, 2025. doi: 10.1145/3689031.3717459.

410 Michael Mitzenmacher. The power of two choices in randomized load balancing. *IEEE Transactions*
411 *on Parallel and Distributed Systems*, 12(10):1094–1104, 2001.

412 Modal. Prefix caching. Documentation, 2026. Accessed 2026-01-28.

413 Zaifeng Pan, Ajikumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin,
414 Yida Wang, and Yufei Ding. KVFlow: Efficient prefix caching for accelerating LLM-based
415 multi-agent workflows. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
416 arXiv:2507.07400.

417 Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo
418 Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *ACM/IEEE*
419 *International Symposium on Computer Architecture (ISCA)*, 2024.

420 Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yanzhe Wu, Weimin Zheng, Xinran Chen,
421 Rui Xu, Xingda Yao, Rongxin Wei, Zhiyuan Zhao, Jing Huang, Mingjie Yang, and Jidong Wu.
422 Mooncake: A KVCache-centric disaggregated architecture for LLM serving, 2024.

423 Qwen Team. Qwen3 technical report. 2025. Alibaba Cloud.

424 Ingo Rechenberg. Evolutionsstrategie: Optimierung technischer systeme nach prinzipien der biolo-
425 gischen evolution. 1973.

426 Vikranth Srivatsa, Sumanth Madhavan, Tian Hu, Sarah Tarun, Yuhan Cheng, Wei Han, Jingyu Yu,
427 Roberto Cristian Rusti, Lifeng Wang, Yiyang Liu, and Hao Zhang. Preble: Efficient distributed
428 prompt scheduling for LLM serving. In *International Conference on Learning Representations*
429 *(ICLR)*, 2025.

430 Biao Sun, Ziming Huang, Hanyu Tang, Chengquan Wang, Cheng Zhang, Yiming Li, Liu Yang,
431 Wencong Zhang, Lin-Wei Wang, Tianhe Wu, Ang Cao, Yongqiang Lin, et al. Llumnix: Dynamic
432 scheduling for large language model serving. In *USENIX Symposium on Operating Systems Design*
433 *and Implementation (OSDI)*, 2024.

434 Fangzhou Wu and Sandeep Silwal. Efficient training-free online routing for high-volume multi-LLM
435 serving. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

436 Tian Xia, Ziming Mao, Jamison Kerney, Ethan J. Jackson, Zhifei Li, Jiarong Xing, Scott Shenker,
437 and Ion Stoica. SkyWalker: A locality-aware cross-region load balancer for LLM inference. arXiv
438 preprint arXiv:2505.24095, 2025.

439 Dongsheng Yang, Austin Li, Kai Li, and Wyatt Lloyd. Learned prefix caching for efficient LLM
440 inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025a.

441 Jingbo Yang, Bairu Hou, Wei Wei, Yujia Bao, and Shiyu Chang. KVLink: Accelerating large
442 language models via efficient KV cache reuse. *arXiv preprint arXiv:2502.16002*, 2025b.

443 Zongheng Yang et al. SkyPilot: An intercloud broker for sky computing. Preprint, 2023.

444 Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Peng, Hao Zhang, Ion Stoica, Kurt Keutzer,
445 and Michael W. Mahoney. CacheBlend: Fast large language model serving for RAG with cached
446 knowledge fusion, 2024.

447 Gyeong-In Yu, Jinseok Jeong, Gyeongchan Kim, Suhyun Kim, and Byung-Gon Chun. Orca: A
448 distributed serving system for transformer-based generative models. In *Proceedings of the 16th*
449 *USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022.

450 Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. WildChat: 1M
451 ChatGPT interaction logs in the wild. In *International Conference on Learning Representations*
452 *(ICLR)*, 2024.

453 Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao
454 Zhuang, Zhuohan Li, Zi Lin, Eric P. Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. LMSYS-
455 Chat-1M: A large-scale real-world LLM conversation dataset. In *International Conference on*
456 *Learning Representations (ICLR)*, 2024a.

- 457 Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi
458 Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang:
459 Efficient execution of structured language model programs. In *Advances in Neural Information*
460 *Processing Systems (NeurIPS)*, 2024b.
- 461 Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao
462 Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language
463 model serving. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*,
464 2024.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract claims (i) public datasets underrepresent long-context multi-turn traffic (quantified in §3), (ii) we evaluate routing policies on a production trace (Tables 2, 3, 4, 5), and (iii) GORGO variants together sweep every TTFT percentile on the W1 and W2 windows; we honestly report that on the daytime stress window `gorgo-hillclimb`’s p99 inflates and `prefix-cache` wins p99 instead. We are explicit in the introduction and §8 that “GORGO” refers to a family of variants and that the result is regime-dependent (the WildChat replay reverses the ranking).

2. Limitations

Question: Does the paper discuss the limitations of the work?

Answer: [Yes]

Justification: Section 8 groups limitations into statistical scope (single trace, single run per window with derived p99 standard error), operational scope (single-tenant fleet, centralized-proxy SPOF), methodological caveats (`gorgo-autotune` identifiability failure, ES convergence time), and system scope (homogeneous hardware, no PD-disaggregation, no scheduler-internal signals).

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete proof?

Answer: [N/A]

Justification: The paper presents an additive cost model and an online optimizer with no formal theorems.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results?

Answer: [Yes]

Justification: Section 6 specifies model, regions, hardware, fleet topology, concurrency, trace windows, and per-window GORGO seeding. Section 4 gives Equation 1, the (1+1)-ES configuration ($\sigma_0 = 0.5$, 1/5-rule, hop 16, window 64), and the median-of-rates fit. The reproducibility statement enumerates four canonical paths in the released artifact.

5. Open access to data and code

Question: Does the paper provide open access to the data and code?

Answer: [No]

Justification: The GLM-5.1 production trace is not redistributable under its terms of service. The proxy and policy code is not yet open-sourced at submission time. The public-dataset characterization (LMSYS, WildChat) is fully reproducible with no proprietary inputs.

6. Experimental setting/details

Question: Does the paper specify all the training and test details?

Answer: [Yes]

Justification: Section 6 specifies model, hardware, regions, concurrency, trace windows, max input tokens, and per-window seeding for adaptive policies. ES configuration is given in Section 4.

7. Experiment statistical significance

Question: Does the paper report error bars or appropriate statistical significance information?

Answer: [No]

Justification: Each data point is a single 30-minute window. Section 8 derives an approximate p99 standard error and shows that the W2 p99 spread between `gorgo-hillclimb` and `random` is inside one standard error. The W1 sweep and W2 p50/p95 wins are several standard errors clear of zero.

8. Experiments compute resources

Question: Does the paper provide sufficient information on compute resources?

Answer: [Yes]

521 Justification: Each policy runs on a dedicated 3-replica fleet of L40S:2 SGLang servers
522 (tensor-parallel 2, 96 GB VRAM per replica). Eight policies $\times$ two windows = 16 fleet-runs,
523 each $\sim$ 30 min wall clock, on three regions. Total GPU-hours: $\approx$ 24.

524 **9. Code of ethics**
525 Question: Does the research conform with the NeurIPS Code of Ethics?
526 Answer: [Yes]
527 Justification: The work concerns routing infrastructure. The production trace is used under
528 terms that permit research replay; we release only aggregated prefix-trie statistics, not
529 individual requests. No individuals are identifiable from the released aggregates.

530 **10. Broader impacts**
531 Question: Does the paper discuss potential societal impacts?
532 Answer: [Yes]
533 Justification: Routing improvements reduce the energy and dollar cost of serving LLMs
534 at fixed quality. Lower-cost serving may accelerate deployment in settings where that is
535 socially harmful; the contributions are infrastructure-level and do not affect model capability.

536 **11. Safeguards**
537 Question: Does the paper describe safeguards for responsible release?
538 Answer: [N/A]
539 Justification: We do not release datasets or models. The released code is a routing proxy
540 and policy library with no inherent dual-use beyond LLM-serving infrastructure.

541 **12. Licenses**
542 Question: Are creators or owners of assets properly credited?
543 Answer: [Yes]
544 Justification: SGLang, vLLM, AIBrix, LMSYS-Chat-1M, WildChat-4.8M, and Qwen3 are
545 cited and used under their published licenses.

546 **13. New assets**
547 Question: Are new assets introduced in the paper well documented?
548 Answer: [Yes]
549 Justification: The proxy, policy library, experiment controller, and trace builder are doc-
550 umented in the repository’s top-level documentation. Spec and manifest files are stored
551 alongside the controller.

552 **14. Crowdsourcing and research with human subjects**
553 Question: Does the paper involve crowdsourcing or human subjects?
554 Answer: [N/A]
555 Justification: No human subjects.

556 **15. IRB approvals**
557 Answer: [N/A]
558 Justification: No human subjects.

559 **16. Declaration of LLM usage**
560 Question: Does the paper describe the usage of LLMs if it is an important, original, or
561 non-standard component of the research methodology?
562 Answer: [N/A]
563 Justification: LLM serving infrastructure is the object of study. Authors used LLMs for
564 routine writing assistance only.

Reproducibility Statement

The experiments are driven by a single experiment controller that launches fleets, configures policies, and replays a trace per policy. The four canonical paths in the released artifact are:

- `specs/policy_matrix_abstract_night.json` — W1 spec (eight policies, manual GORG0 starter weights);
- `specs/policy_matrix_abstract_night_w2.json` — W2 spec (eight policies, GORG0 seeded with W1-learned weights);
- `experiment_runner/policy_matrix_app.py` — experiment controller;
- `data_processing/build_mooncake_trace.py` — trace builder.

W1 and W2 manifests, result CSVs, and summary plots are produced by running the controller against these specs and post-processing per-policy stats with the released analysis scripts. The GLM-5.1 production trace is not redistributable; the public-dataset characterization (LMSYS, WildChat) reproduces with no proprietary inputs.

A WildChat replay (regime-sensitivity control)

We replay a 30-minute window of WildChat-4.8M [Zhao et al., 2024] on the same 2×L40S three-region fleet, with the same eight policies as the GLM-5.1 sweeps. WildChat averages 2,925 input tokens per request and 5.3% intra-user prefix reuse—an order of magnitude shorter than GLM-5.1 and roughly a tenth the reuse—putting it well outside the regime characterized in §3.

Table 6: WildChat-4.8M replay: 30-minute window, $c=32$. TTFT in seconds. Bold is best in column. Compared to the GLM-5.1 results, GORG0 variants are not competitive: `gorgo-static` and `gorgo-autotune` are the worst two policies on p95.

Policy	Completed	p50	p95	p99	max	Succ.%
simple-session-affinity	20,447	0.416	1.025	1.712	7.75	99.6
least-request	20,517	0.480	1.036	1.731	110.45	100.0
random	20,436	0.416	1.077	1.796	15.82	99.6
prefix-cache	20,504	0.497	1.186	2.588	10.63	99.9
least-load	20,484	0.532	1.217	2.535	8.28	99.8
gorgo-hillclimb	20,473	0.541	1.325	2.654	7.05	99.8
gorgo-static	20,462	0.709	1.932	3.969	60.18	99.7
gorgo-autotune	20,451	0.541	3.583	6.438	12.55	99.7

The ranking is reversed: `simple-session-affinity` wins p95 and p99 by a large margin, `least-request` is competitive, `gorgo-hillclimb` sits mid-pack, and `gorgo-static` and `gorgo-autotune` are the worst two policies. Two mechanisms drive this. First, the prefill term in Equation 1 is small at 2,925 tokens; the cost-model weights are mostly fitting noise. Second, intra-user reuse is 5.3% rather than 53%; even a perfect cache-routing decision saves only a few hundred tokens per request, well inside the cross-region RTT band, so chasing cache locality is counterproductive when paired with a cross-region routing penalty. The `gorgo-static` weights frozen from a GLM-5.1 W1 are particularly mis-calibrated to this regime—a reminder that the cost-model parameters do not transfer across workload classes.

We include this result as a regime-sensitivity control, not as a contribution. The takeaway is the same one stated in §3: the gain from joint cache–network–queue routing depends on the workload having long prompts and substantial intra-user prefix reuse. Public chat datasets do not exercise this regime.